

Lab 04 – Constants, Enums, Exceptions, LINQ

Title: Advanced Language Features – Constants, Exceptions, and LINQ

Objectives

This lab aimed to introduce more advanced C# features that make applications more robust and maintainable. I needed to use constants for business rules, implement proper exception handling, and leverage LINQ for more complex data queries. The goal was to make the application resilient to user errors and easier to extend.

Implementation

Updated Angajat Class with Constants

```
public class Angajat
{
    // ... properties ...

    private const double PROCENT_IMPOZIT = 0.25; // 25% total deductions

    // Updated SalariuNet to use the constant
    public double SalariuNet => SalariuBrutEfectiv * (1 - PROCENT_IMPOZIT);

    // ... rest of class ...
}
```

Extended Interface with New Search Method

```
public interface IStocareData
{
    void AddAngajat(Angajat angajat);
    List<Angajat> GetAngajati();
    List<Angajat> CautaDupaNume(string nume);
    List<Angajat> CautaDupaFunctie(string functie);
    bool ModificaAngajat(int id, Angajat angajatNou);
    bool StergeAngajat(int id);
}
```

Implementation of New Search

```
public List<Angajat> CautaDupaFunctie(string functie)
{
    return _angajati.Where(a =>
        a.Functie.Contains(functie, StringComparison.OrdinalIgnoreCase)
    ).ToList();
}
```

Console Application with Exception Handling

```
using System;

namespace SalariiConsole
{
    class Program
    {
        static void Main(string[] args)
        {
            var admin = new AdministrareAngajatiMemorie();
        }
    }
}
```

```
bool running = true;
```

```
while (running)
```

```
{
```

```
    Console.WriteLine("\n=== Meniu Principal ===");
```

```
    Console.WriteLine("1. Adaugă angajat");
```

```
    Console.WriteLine("2. Afișează toți angajații");
```

```
    Console.WriteLine("3. Caută angajat după nume");
```

```
    Console.WriteLine("4. Caută angajat după funcție");
```

```
    Console.WriteLine("5. Filtrare după salariu net minim");
```

```
    Console.WriteLine("6. Ieșire");
```

```
    Console.Write("Alegeți o opțiune: ");
```

```
try
```

```
{
```

```
    string input = Console.ReadLine();
```

```
    if (!int.TryParse(input, out int optiune))
```

```
    {
```

```
        throw new ArgumentException("Opțiunea trebuie să fie un număr.");
```

```
    }
```

```
switch (optiune)
```

```
{
```

```
    case 1:
```

```
        AdaugaAngajat(admin);
```

```
        break;
```

```
    case 2:
```

```
        AfiseazaAngajati(admin);
```

```
        break;
```

```
    case 3:
```

```

        CautaDupaNume(admin);

        break;

    case 4:

        CautaDupaFunctie(admin);

        break;

    case 5:

        FiltrareSalariu(admin);

        break;

    case 6:

        running = false;

        break;

    default:

        Console.WriteLine("Opțiune invalidă.");

        break;

    }

}

catch (FormatException ex)

{

    Console.WriteLine($"Eroare de format: {ex.Message}");

}

catch (ArgumentException ex)

{

    Console.WriteLine($"Eroare: {ex.Message}");

}

catch (Exception ex)

{

    Console.WriteLine($"Eroare neașteptată: {ex.Message}");

}

}

}

```

```

static void AdaugaAngajat(IStocareData admin)
{
    try
    {
        Console.Write("Nume: ");

        string nume = Console.ReadLine();

        Console.Write("Prenume: ");

        string prenume = Console.ReadLine();

        Console.Write("Funcție: ");

        string functie = Console.ReadLine();

        Console.Write("Departament (IT, Marketing, Finante, HR, Productie, Logistica): ");

        string departamentStr = Console.ReadLine();

        if (!Enum.TryParse<Departament>(departamentStr, true, out Departament
departament))
        {
            throw new ArgumentException("Departament invalid.");
        }

        Console.Write("Tip salariu (Lunar/Orar): ");

        string tipStr = Console.ReadLine();

        if (!Enum.TryParse<TipSalariu>(tipStr, true, out TipSalariu tipSalariu))
        {
            throw new ArgumentException("Tip salariu invalid.");
        }

        Console.Write("Salariu brut (pentru Lunar) sau Tarif oră (pentru Orar): ");

        if (!double.TryParse(Console.ReadLine(), out double salariuBrut))
    
```

```
{  
    throw new FormatException("Salariul brut trebuie să fie un număr.");  
}
```

```
Console.Write("Ore lucrate (pentru Orar): ");  
if (!int.TryParse(Console.ReadLine(), out int oreLucrate))  
{  
    oreLucrate = 0;  
}
```

```
Console.Write("Bonusuri: ");  
if (!double.TryParse(Console.ReadLine(), out double bonusuri))  
{  
    bonusuri = 0;  
}
```

```
var angajat = new Angajat  
{  
    Nume = nume,  
    Prenume = prenume,  
    Functie = functie,  
    Departament = departament,  
    TipSalariu = tipSalariu,  
    SalariuBrut = salariuBrut,  
    TarifOra = salariuBrut, // Reuse for simplicity  
    OreLucrate = oreLucrate,  
    Bonusuri = bonusuri,  
    DataAngajarii = DateTime.Today  
};
```

```

        admin.AddAngajat(angajat);

        Console.WriteLine($"Angajat adăugat cu ID: {angajat.Id}");
    }
    catch (FormatException ex)
    {
        Console.WriteLine($"Eroare de format: {ex.Message}");
    }
    catch (ArgumentException ex)
    {
        Console.WriteLine($"Eroare: {ex.Message}");
    }
}

static void FiltrareSalariu(ISTocareData admin)
{
    try
    {
        Console.Write("Introduceți salariul net minim: ");
        if (!double.TryParse(Console.ReadLine(), out double pretMin))
        {
            throw new FormatException("Trebuie să introduceți un număr.");
        }

        var angajati = admin.GetAngajati();
        var filtrati = angajati.Where(a => a.SalariuNet > pretMin).ToList();

        Console.WriteLine($"
Angajați cu salariu net > {pretMin:F2} RON:");

        foreach (var a in filtrati)
        {
            Console.WriteLine(a.Info());
        }
    }
}

```

```

    }

    if (filtrati.Count == 0)
    {
        Console.WriteLine("Niciun angajat nu îndeplinește criteriile.");
    }
}

catch (FormatException ex)
{
    Console.WriteLine($"Eroare: {ex.Message}");
}
}
}
}

```

Why I Used This Approach

Constants: Using `const` for `PROCENT_IMPOZIT` makes the tax rate a compile-time constant. This is appropriate for values that should never change at runtime. It also makes the code more maintainable – if the tax rate changes, I only update one line.

Exception Handling with try-catch: I wrapped user input parsing in try-catch blocks to handle different exception types:

- `FormatException`: For non-numeric input
- `ArgumentException`: For validation issues like invalid enum values
- `Exception`: A catch-all for unexpected errors

This ensures the application never crashes due to bad user input.

LINQ with Lambda Expressions: The salary filter uses `Where` with a lambda expression: `a => a.SalariuNet > pretMin`. This is concise and expressive, clearly communicating the filtering intent.

Enum.TryParse: This method safely parses a string to an enum value without throwing exceptions. It's the preferred way to convert strings to enums in modern C#.

Testing & Results

I tested the exception handling by entering invalid data:

- Entering "abc" for a number field caught the `FormatException` and displayed a friendly error
- Entering an invalid department name caught the `ArgumentException`
- The salary filter correctly showed only employees meeting the criteria

Reflection

This lab showed me that real applications need to be resilient. Users will make mistakes, and the application should handle them gracefully. The combination of constants, exception handling, and LINQ makes the code both robust and expressive. I'm starting to appreciate how these features work together to create professional-quality software. The distinction between different exception types allows for more targeted error handling.